

Unity Quest System Documentation

This document explains how to use the quest system in Unity, how the parts connect, how to create multiple quests, how to test them in a scene, and how to avoid common mistakes. The system is built around ScriptableObjects for static authored quest data, runtime classes for player progress, and a manager/service approach for orchestration, which matches Unity guidance on separating static data from gameplay logic and runtime state.[cite:18][cite:46][cite:121]

System overview

The quest system has three main layers:

- **Authoring layer:** QuestDefinitionSO, QuestDatabaseSO, node definitions, branch definitions, and reward definitions. These are assets created in the Project window and edited in the Inspector.[cite:46][cite:121]
- **Runtime layer:** QuestRuntimeState, QuestNodeRuntimeState, and QuestJournalState. These track accepted quests, current node, revealed objectives, completion, and tracking state while the game runs.[cite:18]
- **Scene/integration layer:** QuestManager, NpcQuestGiver, DialogueQuestBridge, reporter scripts, and UI presenters. These are the MonoBehaviours placed in the scene to make the system work.[cite:18][cite:73]

Think of the system like this:

- QuestDefinitionSO = one quest
- QuestDatabaseSO = a container of many quests
- QuestManager = the runtime brain that accepts, tracks, advances, completes, and saves quests
- NPCs and gameplay systems = send events to the manager
- UI = reads the manager state and shows the current quest information

Core idea

The most important rule is this: **do not store player progress inside the quest asset itself.** ScriptableObjects are best used as data containers for authored definitions, while mutable progress should live in runtime/save data.[cite:18][cite:46][cite:121]

That means:

- Quest asset stores title, start NPC, steps, branches, rewards, prerequisites.
- Runtime state stores current step, accepted/completed state, chosen branch, progress counts, and tracked quest.

- Save file stores the runtime state, not the quest asset.

Multiple quests

A new database asset is **not** needed for every new quest. The normal setup is one QuestDatabaseSO that contains a list of many QuestDefinitionSO assets.[cite:111][cite:118]

Example:

- Q_FindHerbs.asset
- Q_FarmerApples.asset
- Q_CaveWarning.asset
- all three are added to QuestDatabase_Main.asset

Then QuestManager references only this one database:

```
QuestManager
- questDatabase -> QuestDatabase_Main
```

When a new quest is needed later:

1. Create a new QuestDefinitionSO asset.
2. Fill in its fields.
3. Add it to the quests list in QuestDatabase_Main.
4. If needed, create an NPC with a matching NpcIdentity and NpcQuestGiver.

No code changes are required just to add a new quest if the quest fits the current system.

Accepted quest vs tracked quest

The system supports:

- **Accepted quests:** quests the player has already started
- **Tracked quest:** the one currently highlighted in the UI

The player can accept many quests but track only one at a time. Switching the tracked quest does **not** reset the other accepted quests. Their runtime progress remains in QuestJournalState.[cite:17][cite:12]

Example:

- Player accepts quest_find_herbs
- Player finishes step 1
- Player accepts quest_farmer_apples
- Player tracks quest_farmer_apples
- quest_find_herbs still keeps its previous progress and can be resumed later

Scene setup

A minimal scene for testing should contain these objects:

```
Scene
├── QuestBootstrap
│   ├── QuestManager
│   ├── QuestRewardInstaller
│   ├── DialogueQuestBridge
│   ├── GrantXpRewardHandler
│   ├── GrantItemRewardHandler
│   ├── PlayCutsceneRewardHandler
│   ├── UnlockQuestRewardHandler
│   └── UnlockLevelRewardHandler
├── HealerNPC
│   ├── NpcIdentity
│   ├── NpcQuestGiver
│   └── optional test script
├── FarmerNPC
│   ├── NpcIdentity
│   ├── NpcQuestGiver
│   └── optional test script
├── HerbTester
│   ├── QuestCollectReporter
│   └── optional test script
└── Canvas
    ├── QuestMenu
    │   ├── LeftPanel
    │   │   └── Scroll View
    │   └── RightPanel
    └── QuestUIControllers
        ├── QuestListPresenter
        └── QuestDetailsPresenter
```

This structure uses Unity UI's `ScrollRect` pattern for the quest list, which is the standard approach for scrollable UI content.[cite:73][cite:95]

Creating a quest asset

To create a quest:

1. In the Project window, right click.
2. Select **Create > Quest System > Quest Definition**.
3. Name it something like `Q_FindHerbs`.
4. Fill these fields:
 - `questId`
 - `questTitle`

- `shortDescription`
- `startNPCId`
- `startNodeId`
- `nodes`
- `prerequisiteQuestIds` if needed
- `rewards`

A very small test quest can look like this:

Quest identity

- `questId = quest_find_herbs`
- `questTitle = Find Herbs`
- `shortDescription = Help the healer by gathering herbs.`
- `startNPCId = npc_healer_01`
- `startNodeId = node_talk_healer`

Nodes

Node ID	Type	Target ID	Required	Next	Purpose
<code>node_talk_healer</code>	TalkToNpc	<code>npc_healer_01</code>	1	<code>node_collect_herb</code>	Start quest step
<code>node_collect_herb</code>	CollectItem	<code>herb</code>	3	<code>node_return_healer</code>	Collect herbs
<code>node_return_healer</code>	TalkToNpc	<code>npc_healer_01</code>	1	empty / terminal	Finish quest

Creating multiple quests

To add a second quest, repeat the same process with a new asset.

Example second quest:

`Q_FarmerApples`

- `questId = quest_farmer_apples`
- `questTitle = Gather Apples`
- `shortDescription = Bring apples to the farmer.`
- `startNPCId = npc_farmer_01`
- `startNodeId = node_talk_farmer`

Nodes:

Node ID	Type	Target ID	Required	Next
node_talk_farmer	TalkToNpc	npc_farmer_01	1	node_collect_apple
node_collect_apple	CollectItem	apple	2	node_return_farmer
node_return_farmer	TalkToNpc	npc_farmer_01	1	terminal

Then add both quests to the **same** database asset:

```
QuestDatabase_Main
- quests [2]
  0 -> Q_FindHerbs
  1 -> Q_FarmerApples
```

Creating the quest database

To create the database:

1. Right click in the Project window.
2. Select **Create > Quest System > Quest Database**.
3. Name it QuestDatabase_Main.
4. Set the quests list size.
5. Drag all QuestDefinitionSO assets into the list.

This database is then assigned once to QuestManager. That database asset acts as the single reference point for all available quest definitions in the game.[cite:111][cite:121]

Setting up QuestManager

Create an empty GameObject called QuestBootstrap and add:

- QuestManager
- QuestRewardInstaller
- DialogueQuestBridge
- reward handlers you want to test

Assign on QuestManager:

- questDatabase = QuestDatabase_Main
- autoLoadOnAwake = true
- autoSaveOnChange = true

Assign on QuestRewardInstaller:

- questManager = QuestBootstrap
- rewardHandlerBehaviours = list of reward handler components

Because the runtime journal is saved using a JSON repository, the quest state can be stored under `Application.persistentDataPath`, which Unity documents as the appropriate place for data retained between runs.[cite:69]

Setting up NPC quest givers

Each quest-giving NPC needs at minimum:

- `NpcIdentity`
- `NpcQuestGiver`

Example NPC setup:

Healer NPC

- `NpcIdentity.npcId = npc_healer_01`
- `NpcQuestGiver.questId = quest_find_herbs`
- `NpcQuestGiver.questManager = QuestBootstrap`

Farmer NPC

- `NpcIdentity.npcId = npc_farmer_01`
- `NpcQuestGiver.questId = quest_farmer_apples`
- `NpcQuestGiver.questManager = QuestBootstrap`

Important rule:

- `QuestDefinitionSO.startNpcId` must exactly match `NpcIdentity.npcId`

If they do not match exactly, the quest will not be accepted from that NPC.

Setting up the UI

The quest UI uses a left/right layout:

- **Left panel:** accepted quest list in a scrollable `ScrollView`
- **Right panel:** selected quest title, description, and only the current visible objective

Unity's `ScrollView` requires a content object and a viewport, and can optionally use layout components to support dynamic lists.[cite:73][cite:95]

UI hierarchy example

```
Canvas
├── QuestMenu
│   ├── LeftPanel
│   │   └── QuestScrollView
│   │       ├── Viewport
│   │       └── Content
│   └── RightPanel
```

```
├─ QuestTitleText
├─ QuestDescriptionText
└─ CurrentObjectiveText
```

Components needed

On QuestScrollView

- Image
- ScrollRect

On Viewport

- Image
- Mask

On Content

- Vertical Layout Group
- Content Size Fitter

Then assign in the ScrollRect:

- Viewport = Viewport
- Content = Content
- Horizontal = off
- Vertical = on

This is the standard Unity setup for vertical scrolling UI content.[cite:73][cite:95]

Setting up the presenters

Add QuestListPresenter and QuestDetailsPresenter to a controller object in the Canvas.

QuestListPresenter

Assign:

- questManager
- contentRoot
- itemPrefab
- detailsPresenter

QuestDetailsPresenter

Assign:

- questManager
- questTitleText
- questDescriptionText
- currentObjectiveText

The list presenter rebuilds the left list from accepted quests. The details presenter shows the selected quest and only the current objective, which matches the requirement of hiding future quest steps from the player until they are revealed.

Testing in scene

A simple manual test flow:

1. Press Play.
2. Accept quest from NPC.
3. Verify quest appears in left list.
4. Verify right panel shows the first objective.
5. Simulate or perform the required gameplay action.
6. Verify the right panel updates to the next objective.
7. Finish the final objective.
8. Verify the quest completes.

For quick testing before full gameplay integration, temporary debug scripts can call quest events directly.

Example temporary test keys:

- E = try to accept quest
- T = report NPC talk
- H = report herb collected

This is only for debug testing and should later be replaced by the real dialogue and gameplay systems.

Branching quests

The system supports branching by:

- dialogue choice
- gameplay outcome
- automatic branch resolution

A branch is defined on a node using QuestBranchDefinition:

- `conditionType`
- `conditionKey`
- `conditionValue`
- `nextNodeId`
- `failQuestIfMatched`

Example:

- `node node_return_healer`
- `branch A: dialogue choice give_herbs_kindly -> node_good_end`
- `branch B: dialogue choice refuse_help -> fail quest`

The branch resolver decides which path to take based on the event payload reaching the quest manager.

Rewards

Rewards are data-driven and extensible. That means the quest asset stores reward definitions, and the reward processor dispatches them to registered handlers. ScriptableObjects are good for holding reward configuration data, while code performs the actual game behavior.[cite:46]
[cite:18]

Built-in reward types currently include:

- `GrantXP`
- `GrantItem`
- `PlayCutscene`
- `UnlockQuest`
- `UnlockLevel`
- `Custom`

To add a new reward type later:

1. Add a new value to `QuestRewardType`.
2. Create a new handler implementing `IQuestRewardHandler`.
3. Register the handler using `QuestRewardInstaller`.
4. Use the new type inside quest reward definitions.

This avoids rewriting `QuestManager` every time a new reward type is introduced.

Save system behavior

The current quest runtime state is saved to JSON through `JsonQuestStateRepository`, which uses `Application.persistentDataPath`. Unity documents this path as a persistent directory for data you want to retain between runs.[cite:69]

The save file stores runtime state such as:

- accepted quests
- current node ID
- tracked quest ID
- node completion state
- chosen branch
- quest status

The save file does **not** store the authored quest definitions, because those are already contained in the ScriptableObject assets.

Common mistakes

1. Trailing spaces in IDs

This is one of the easiest bugs to create and one of the hardest to notice.

Bad:

- `node_talk_healer`

Good:

- `node_talk_healer`

All of these must match exactly:

- `startNodeId`
- each node's `nodeId`
- each node's `nextNodeId`
- NPC IDs
- target IDs used by reporters

If IDs do not match exactly, quest progression can fail because the system uses exact string matching.

2. Quest is not in database

If a quest asset is not added to `QuestDatabaseSO.quests`, the manager cannot find it.

3. NPC ID mismatch

If `NpcIdentity.npcId` does not equal the quest's `startNpcId`, that NPC will not be able to offer the quest.

4. Reporter target mismatch

If the collect reporter sends `apple` but the quest step expects `herb`, that step will never complete.

5. Old broken save file

If the save file contains outdated runtime data from an earlier broken version, the quest may keep failing until the old JSON file is deleted and the test is rerun.[cite:69]

Recommended workflow for adding a new quest

Use this checklist whenever a new quest is added:

1. Create `QuestDefinitionSO` asset.
2. Fill quest identity fields.
3. Create nodes with clean IDs.
4. Set `startNodeId`.
5. Add rewards if needed.
6. Add prerequisite quest IDs if needed.
7. Add quest asset to `QuestDatabase_Main`.
8. Create or configure NPC with matching `NpcIdentity`.
9. Connect `NpcQuestGiver.questId`.
10. Test acceptance and progression in play mode.
11. Check UI updates.
12. If needed, delete the old JSON save and retest.

Example full multi-quest setup

Database

`QuestDatabase_Main`

- `Q_FindHerbs`
- `Q_FarmerApples`
- `Q_CaveWarning`

NPCs

NPC object	NPC ID	Gives quest
HealerNPC	<code>npc_healer_01</code>	<code>quest_find_herbs</code>
FarmerNPC	<code>npc_farmer_01</code>	<code>quest_farmer_apples</code>

NPC object	NPC ID	Gives quest
GuardNPC	npc_guard_01	quest_cave_warning

Result

When the scene starts, QuestManager knows all available quests from the database. Each NPC can offer a different quest based on its `questId`, and the UI can list all accepted quests while showing only one tracked quest at a time.

Future improvements

Useful future upgrades:

- Trim and sanitize IDs in code to avoid whitespace bugs.
- Build a dictionary cache inside QuestDatabaseSO for faster quest lookup by ID.
- Add editor validation that warns when duplicate quest IDs or node IDs exist.
- Integrate the quest save into a shared save framework for inventory, dialogue, and progression systems.
- Replace temporary debug input with the real dialogue and pickup systems.

Quick reference

Create a new quest

- Create QuestDefinitionSO
- Fill data
- Add to QuestDatabase_Main
- Link to NPC with matching ID

Add another quest

- Do not create a new database just for that quest
- Add the new quest asset into the same database list

Test a quest

- Accept from NPC
- Send step events
- Watch left list and right panel
- Check `quest_journal.json` if needed

If quest is not progressing

Check in this order:

1. Quest is in database
2. IDs have no extra spaces
3. NPC ID matches start NPC ID
4. Target ID matches reporter/event ID
5. Save file is not stale

Main rule

Static data belongs in ScriptableObjects. Mutable player progress belongs in runtime/save state.
[cite:18][cite:46][cite:121]